

Advanced Data Science Program

Assignment 01 Python Data Structures

Job Market Skill Analyzer

Assignment Task - Python Data Structures

Project Title - Job Market Skill Analyzer

Problem Statement

A recruitment analytics team wants a compact Python tool to analyze skills requested across job postings for data and analytics roles. The dataset contains basic job posting information (role, company, and a single text field of skills). Your task is to create a Python program (Colab or Jupyter Notebook) that stores job posting data, allows adding new postings, extracts and analyzes skills, and saves cleaned/structured output to a file.

Dataset:

Start your notebook with the following dictionary (use exactly this variable name):

```
job_data = { "Job_ID": [1, 2, 3, 4, 5],  
             "Role": ["Data Analyst", "Data Scientist", "Business Analyst", "Data Engineer", "ML  
Engineer"],  
             "Company": ["ABC Corp", "XyZ Ltd", "DataWorks", "InfraTech", "AI Labs"],  
             "Skills": [  
                 "Excel SQL Python",  
                 "Python ML Statistics",  
                 "Excel SQL PowerBI",  
                 "Python SQL Spark",  
                 "Python ML DeepLearning"  
             ]}  
  
print(job_data)
```

Task:

1) Using a for loop, print each job posting in a readable format.

Example Output

ID: 1 | Role: Data Analyst | Company: ABC Corp | Skills: Excel SQL Python

ID: 2 | Role: Data Scientist | Company: XyZ Ltd | Skills: Python ML Statistics

2) Add New Job Postings (interactive)

Ask the user how many job postings they want to add. For each posting, collect the **role**, **company**, and **skills** (skills provided as a single string, space-separated or comma-separated).

Requirements:

- **Job_ID** should auto-increment from the maximum existing ID.
- Convert numeric inputs (if any) to integers where applicable.
- Append new values to the lists in **job_data**.

Example interaction

How many job postings do you want to add? 2

Enter role: Junior Data Analyst

Enter company: BrightData

Enter skills (space-separated or comma-separated): Excel, SQL, Python

3) Clean & Normalize Skills (text processing)

Make a function called `clean_skills()` that accepts skills in string format.

- Converts text to lowercase
- Replaces commas with spaces
- Removes extra spaces (normalize whitespace)
- Optionally removes punctuation (you may use basic `str.replace()` or `re` module)

Apply `clean_skills()` to every entry in `job_data['Skills']` and overwrite the list with cleaned versions.

4) Skill Extraction & Frequency

- From the cleaned skills list, split each skill string into individual skill tokens.
- Build a consolidated list of all skills that contains every skill token across all postings.
- Use a dictionary or other structure to compute `skill_count = {skill: frequency}`.
- Print the top 5 most frequent skills (or all if < 5).

Expected print example:

Top skills by frequency:

Python: 5

sql: 4

Excel: 2

ML: 2

Power BI: 1

5) Basic Statistics & Insights

Calculate and print:

- Total number of job postings
- Number of unique skills
- Jobs that mention "python" (case-insensitive)

Sample Output

Total Jobs: 7

Unique skills: 9

Jobs mentioning Python: ['Data Analyst', 'Data Scientist', 'Data Engineer', ...]

6) Skill Search Function (functions & conditional logic)

Create a function, **find_jobs_by_skill(skill_query)**, that returns a list of (**Job_ID**, **Role**, **Company**) for postings containing the skill_query token.

- The function must normalize the skill_query (lowercase, strip spaces)
- Use if/elif/else where appropriate

Example: find_jobs_by_skill('sql') → returns matching jobs.

7) Lambda Filtering & List Comprehension

Use a lambda function and filter() (or a list comprehension with a lambda-like expression) to extract job roles where the skill count (from skill_count) for any skill ≥ 2 — that is, roles that include at least one relatively common skill.

Example Output

Roles with at least one common skill: ['Data Analyst', 'Data Scientist', 'ML Engineer']

8) Unique Skill Set

Convert the consolidated skill list into a set and print the unique skills in alphabetical order.

Example

Unique skills:

```
['deepLearning', 'excel', 'ml', 'powerbi', 'python', 'spark', 'sql', 'statistics']
```

9) Save Cleaned Data to File (file handling & exceptions)

Save the cleaned & structured job records to a file named **job_skills.csv**. Each line should contain:

Job_ID,Role, Company, Skills

1,Data Analyst,ABC Corp,excel sql python

Use try/except/else/finally to handle file I/O errors. Also, create a secondary file, top_skills.txt that lists top skills and counts.

Deliverables

Students must submit a Google Drive link containing the following:

1. **Python Notebook** (Google Colab or Jupyter Notebook) with the complete code and outputs visible.
 - If using Google Colab, share the notebook link with **View** access.
 - If using Jupyter: upload the notebook to Google Drive and share the folder with **View** access.
2. **Generated files:** job_skills.csv and top_skills.txt (these should be created by the notebook and listed in the Drive folder).
3. Place the notebook, generated files, and summary document in a single Google Drive folder and share the folder (view access).

Skills to be Evaluated

Assignment Section	Core Skill	Sub-Skill	Expected Proficiency
Data Setup & Load	Python Basics	Lists, Dictionaries, Appending	Beginner
Adding Postings	Input & Looping	input(), append(), loop control	Beginner
Cleaning Skills	Text Processing	Lowercasing, punctuation removal, whitespace normalization	Intermediate
Skill Extraction	Logical/Analytical Skills	Splitting, counting, and dictionary operations	Intermediate
Summary Statistics	Data Analysis	Unique count, filtering	Intermediate
Functions & Search	Code Structure	Function, parameters, return values	Beginner
Lambda & Filters	Functional Constructs	lambda, filter, list comp	Beginner
File Handling	I/O & Exceptions	write(), try/except	Beginner

Rubric & Mark Allocation (Total: 10 Marks)

Skill Component	Marks	Full Marks Criteria	Partial Credit Description	Partial Credit (Rounded Range)
Data Setup & Display	1	job_data present and correctly printed in a readable format	Data is present, but formatting or printing has minor issues	0–1
Adding Postings (interactive)	2	Inputs work, Job_ID auto-increments, new postings appended correctly	Some additions appended, but ID or type casting issues	0–1
Cleaning & Normalization Skill Extraction & Frequency	3	clean_skills() implemented and applied to all entries all_skills and skill_count computed correctly, and top skills printed	Cleaning present but inconsistent (e.g., commas removed but extra spaces remain) Counting partially correct or missing sorting/top-k	1–2
Function Implementation (search)	2	find_jobs_by_skill() returns correct job tuples	Function present but returns incomplete results or has edge-case issues	0–1
File Handling & Exceptions	2	job_skills.csv and top_skills.txt were written successfully with try/except	Files written but exception handling is incomplete or missing finally/else	0–1

Notes on Partial Credit:

- Give partial credit proportionally. For example, if a component is attempted and half of the requirements are correct, assign ~50% of that component's marks.
- If the code runs but produces incorrect outputs due to logic errors, award partial marks for effort and structure.
- Missing deliverables (files or summaries) should reduce marks in the File Handling or Reporting part accordingly.

Skill Taxonomy Structure (for evaluators)

Data Loading & Management

- Loading dictionaries/lists
- Adding new entries
- Structuring data properly

Data Cleaning

- Lowercasing, punctuation removal
- Space normalization
- Consistency checks

Functions & Analysis

- User-defined functions
- Loops, conditionals
- Counting, sorting
- Basic statistical measures (counts, unique)

Data Summarization

- Filtering and grouping
- Extracting unique skills

Evaluation Guidelines

1. Run the notebook start-to-finish; ensure there are no runtime errors.
2. Verify `job_data` is present and printed.
3. Add a sample posting to confirm the interactive section works (use a code cell simulation if running non-interactive tests).
4. Inspect cleaned skills values for consistent normalization.
5. Confirm `skill_count` values are correct by manual inspection of smaller inputs.
6. Test `find_jobs_by_skill()` with a known skill (e.g., Python) and verify results.
7. Confirm that the files `job_skills.csv` and `top_skills.txt` are created and contain the expected content.

Scoring approach: Allocate marks per the rubric table. Provide short feedback per component, highlighting strengths and improvements.

Achievement Criteria & Passing Rules

- **Attempt requirement:** Each rubric component must have at least partial credit (not zero). A completely skipped section yields 0 for that component.
- **Minimum competence rule:** Students must fully achieve at least two major components among *cleaning & normalization, skill extraction & frequency, and adding postings/functions & logic to pass*.
- **Feedback:** Provide a detailed scorecard and comments for each component.

Submission checklist for students

- Notebook with code and outputs
- job_skills.csv and top_skills.txt in the Drive folder
- Shared Drive folder link (View access)